PROGRAMACIÓN Y ESTRUCTURA DE DATOS

PRÁCTICA 1

GRUPO 19 | Daniel Atanasov, Raúl Sanz y Nazario Touza. (Grupo de prácticas 3L, viernes 18-20h)

PRACTICA 1 | Gestión de viajes de una empresa de autobuses.

PROFESORA | Nuria Serrano

ÍNDICE

1. Especificación lógica de los TADs diseñados.
    1.1. TAD Autobús
    1.2. TAD Viaje
    1.3. TAD GestionBus

2. Descripción de los procesos llevados a cabo en los métodos que implementan las opciones del menú principal.
    2.1. Opción 1: Menú de Mantenimiento
        2.1.1. Alta de autobús
        2.1.2. Baja de autobús
        2.1.3. Modificación de autobús
        2.1.4. Registrar viaje
        2.1.5. Borrar viaje
        2.1.6. Modificación de viaje
    2.2. Opción 2: Menú de Listados
        2.2.1. Listado general de autobuses por número de plazas ascendente
        2.2.2. Listado general de autobuses por número de plazas descendente
        2.2.3. Listado general de viajes
        2.2.4. Listado de viajes que parten de una ciudad determinada
        2.2.5. Listado de viajes que llegan a una ciudad determinada
        2.2.6. Listado de autobuses que tienen una capacidad mayor o igual a la indicada por el usuario
        2.2.7. Listado del autobús o autobuses que realizan la mayor cantidad de viajes
        2.2.8. Cantidad total de viajes que realizan los autobuses de la empresa
        2.2.9. Cantidad total de pasajeros que viajan de una ciudad a otra

3. Listado del código fuente desarrollado

3.1. Clase Autobus
3.2. Clase Viaje
3.3. Clase GestionAutobus
3.4. Clase LEG
3.5. Clase NodoLEG
3.6. Clase I_LE

4. Pruebas de ejecución

1. ESPECIFICACIÓN LÓGICA DE LOS TADs DISEÑADOS

En esta práctica se han diseñado tres TADs distintos para la gestión de autobuses, viajes y su organización. Dichos TADs se explican a continuación:

1.1. TAD Autobús

- DEFINICIÓN: Este TAD representa un autobús en la empresa. Cada uno se identifica de forma única por su matrícula y mantiene una colección de sus viajes asociados.
- ELEMENTOS: matrícula (no pueden existir dos iguales), el año en que se compró, las plazas disponibles y un listado de los viajes que realiza.
- ESTRUCTURA: Relación no lineal.
- DOMINIO: la matrícula es una cadena no vacía, el año de compra un entero y el número de plazas un entero mayor que 0. El dominio lo forman todas las combinaciones posibles que cumplen estas restricciones.
- OPERACIONES:
    - **Nombre:** crearAutobus
        - **Utilidad:** Crea un nuevo objeto Autobús con los datos proporcionados y la lista de viajes vacía.
        - **Entrada:** String matricula, int anioCompra e int plazas.
        - **Salida:** Autobús.
        - **Precondiciones:** matricula no nula y número de plazas mayor que cero.
        - **Postcondiciones:** se devuelve un autobús con los datos proporcionados y la lista de viajes vacía.
    - **Nombre:** getMatricula
        - **Utilidad:** devuelve la matrícula del autobús.
        - **Entrada:** un objeto de tipo autobús.
        - **Salida:** String.
        - **Precondiciones:** el autobús debe estar inicializado.
        - **Postcondiciones:** devuelve la matrícula del autobús.
    - **Nombre:** getAnioCompra
        - **Utilidad:** devuelve el año de compra del autobús.

- **Entrada:** un objeto de tipo autobús.
- **Salida:** int.
- **Precondiciones:** el autobús debe estar inicializado.
- **Postcondiciones:** devuelve el año de compra del autobús.

- **Nombre:** getPlazas
  - **Utilidad:** devuelve el número de plazas del autobús.
  - **Entrada:** un objeto de tipo autobús.
  - **Salida:** int.
  - **Precondiciones:** el autobús debe estar inicializado.
  - **Postcondiciones:** devuelve el número de plazas del autobús.

- **Nombre:** setAnioCompra
  - **Utilidad:** modifica el año de compra del autobús.
  - **Entrada:** un objeto de tipo autobús y un entero que será el nuevo año.
  - **Salida:** ninguna.
  - **Precondiciones:** el año debe ser válido.
  - **Postcondiciones:** se actualiza el año de compra del autobús.

- **Nombre:** setPlazas
  - **Utilidad:** modifica el número de plazas del autobús.
  - **Entrada:** un objeto de tipo autobús y un entero que será el nuevo número de plazas.
  - **Salida:** ninguna.
  - **Precondiciones:** el número de plazas debe ser mayor que cero.
  - **Postcondiciones:** se actualiza el número de plazas del autobús.

- **Nombre:** equals
  - **Utilidad:** compara dos autobuses para determinar si son iguales según su matrícula.
  - **Entrada:** dos objetos de tipo autobús.
  - **Salida:** boolean.
  - **Precondiciones:** ambos objetos no nulos.
  - **Postcondiciones:** devuelve true si las matrículas coinciden (ignorando mayúsculas/minúsculas) o false en caso contrario.

1.2. TAD Viaje

- DEFINICIÓN: Este TAD representa un viaje realizado por un autobús dentro del sistema. Cada viaje se identifica de forma única mediante un código.
- ELEMENTOS: código identificador (no pueden existir dos iguales), ciudades de origen y destino y hora del viaje.
- ESTRUCTURA: Relación lineal.
- DOMINIO: el código es un entero, el origen y destino son cadenas de caracteres no vacías y distintas entre sí, y la hora es una cadena de caracteres. El dominio lo forman todas las combinaciones posibles que cumplen estas restricciones.
- OPERACIONES:
  - **Nombre:** crearViaje
    - **Utilidad:** Crea un nuevo objeto Viaje con los datos proporcionado.
    - **Entrada:** int código, String origen, String destino y String hora.
    - **Salida:** Viaje.
    - **Precondiciones:** origen y destino no nulos y deben ser distintos.
    - **Postcondiciones:** se devuelve un viaje con los datos proporcionados.
  - **Nombre:** getCodigo
    - **Utilidad:** devuelve el código identificador del viaje.
    - **Entrada:** un objeto de tipo viaje.

- **Salida:** int.
- **Precondiciones:** el viaje debe estar inicializado.
- **Postcondiciones:** devuelve el código del viaje.

- **Nombre:** getOrigen
  - **Utilidad:** devuelve la ciudad desde la que sale el viaje.
  - **Entrada:** un objeto de tipo viaje.
  - **Salida:** String.
  - **Precondiciones:** el viaje debe estar inicializado.
  - **Postcondiciones:** devuelve la ciudad de origen.

- **Nombre:** getDestino
  - **Utilidad:** devuelve la ciudad en la que termina el viaje.
  - **Entrada:** un objeto de tipo viaje.
  - **Salida:** String.
  - **Precondiciones:** el viaje debe estar inicializado.
  - **Postcondiciones:** devuelve la ciudad de destino.

- **Nombre:** getHora
  - **Utilidad:** devuelve la hora a la que sale el viaje.
  - **Entrada:** un objeto de tipo viaje.
  - **Salida:** String.
  - **Precondiciones:** el viaje debe estar inicializado.
  - **Postcondiciones:** devuelve la hora del viaje.

- **Nombre:** setOrigen
  - **Utilidad:** modifica la ciudad desde la que sale el viaje.

- **Entrada:** un objeto de tipo viaje y un String que será la nueva ciudad de origen.
- **Salida:** ninguna.
- **Precondiciones:** origen no nulo ni vacío.
- **Postcondiciones:** se actualiza la ciudad de origen del viaje.

- **Nombre:** setDestino
  - **Utilidad:** modifica la ciudad en la que termina el viaje.
  - **Entrada:** un objeto de tipo viaje y un String que será la nueva ciudad de destino.
  - **Salida:** ninguna.
  - **Precondiciones:** destino no nulo ni vacío.
  - **Postcondiciones:** se actualiza la ciudad de destino del viaje.

- **Nombre:** setHora
  - **Utilidad:** modifica la hora a la que sale el viaje.
  - **Entrada:** un objeto de tipo viaje y un String que será la nueva hora.
  - **Salida:** ninguna.
  - **Precondiciones:** ninguna.
  - **Postcondiciones:** se actualiza la hora de salida del viaje.

- **Nombre:** equis
  - **Utilidad:** compara dos viajes para determinar si son iguales según su código.
  - **Entrada:** dos objetos de tipo viaje.
  - **Salida:** boolean.
  - **Precondiciones:** ambos objetos no nulos.
  - **Postcondiciones:** devuelve true si los códigos coinciden o false en caso contrario.

- **Nombre:** toString
  - **Utilidad:** devuelve una representación en forma de cadena del viaje.
  - **Entrada:** un objeto de tipo viaje.
  - **Salida:** String.
  - **Precondiciones:** el viaje debe estar inicializado.
  - **Postcondiciones:** devuelve una cadena con el formato del viaje (código, origen, destino y hora).

1.3. TAD GestionBus

- DEFINICIÓN: Este TAD representa el sistema de gestión de autobuses de la empresa, permitiendo administrar los autobuses y los viajes asociados mediante distintas operaciones de mantenimiento y consulta.
- ELEMENTOS: colección de autobuses almacenados en una lista enlazada genérica (LEG<Autobus>).
- ESTRUCTURA: Relación lineal de autobuses, donde cada autobús contiene una colección de viajes.
- DOMINIO: contiene objetos de tipo Autobús, cada uno con una matrícula única y una colección de viajes asociados. El dominio lo forman todas las combinaciones posibles de autobuses y viajes que cumplen las restricciones definidas en sus respectivos TADs.
- OPERACIONES:
  - **Nombre:** altaAutobus
    - **Utilidad:** añade un nuevo autobús a la colección.
    - **Entrada:** datos del autobús que se quiere añadir.
    - **Salida:** ninguna.
    - **Precondiciones:** no debe existir un autobús con la misma matrícula.
    - **Postcondiciones:** el autobús se añade a la colección.
  - **Nombre:** bajaAutobus
    - **Utilidad:** elimina un autobús de la colección.
    - **Entrada:** matrícula del autobús que se quiere eliminar.

- **Salida:** ninguna.
- **Precondiciones:** el autobús debe existir.
- **Postcondiciones:** el autobús se elimina de la colección.

- **Nombre:** modificarAutobus
  - **Utilidad:** modifica los datos de un autobús existente.
  - **Entrada:** matrícula y nuevos datos del autobús.
  - **Salida:** ninguna.
  - **Precondiciones:** el autobús debe existir.
  - **Postcondiciones:** se actualizan los datos del autobús.

- **Nombre:** registrarViaje
  - **Utilidad:** añade un viaje a un autobús determinado.
  - **Entrada:** matrícula del autobús y datos del viaje.
  - **Salida:** ninguna.
  - **Precondiciones:** el autobús debe existir y el código del viaje no debe estar repetido.
  - **Postcondiciones:** el viaje queda asociado al autobús.

- **Nombre:** borrarViaje
  - **Utilidad:** elimina un viaje de un autobús.
  - **Entrada:** matrícula del autobús y código del viaje.
  - **Salida:** ninguna.
  - **Precondiciones:** el autobús y el viaje deben existir.
  - **Postcondiciones:** el viaje se elimina de la colección del autobús correspondiente.

- **Nombre:** modificarViaje

- **Utilidad:** modifica los datos de un viaje existente.
- **Entrada:** matrícula del autobús, código y nuevos datos del viaje.
- **Salida:** ninguna.
- **Precondiciones:** el autobús y el viaje deben existir.
- **Postcondiciones:** se actualizan los datos del viaje.

2. DESCRIPCIÓN DE LOS PROCESOS LLEVADOS A CABO EN LOS MÉTODOS QUE IMPLEMENTAN LAS OPCIONES DEL MENÚ PRINCIPAL

En este apartado se explican los procesos que se ejecutan en cada una de las opciones del menú principal, detallando las operaciones realizadas sobre las estructuras de datos del sistema.

2.1. Opción 1: Menú de Mantenimiento

2.1.1. Alta de autobús

<u>Método:</u> altaAutobús()

El programa solicita al usuario la matrícula del nuevo autobús y comprueba si ya existe en la lista, empleando el método buscar(). Si no existe, el programa solicita el resto de datos y crea el objeto Autobus. La inserción se realiza de forma ordenada según el número de plazas, recorriendo la lista y enlazando el nuevo nodo en su posición correspondiente.

2.1.2. Baja de autobús

<u>Método:</u> bajaAutobus()

El programa solicita al usuario la matrícula del autobús que se desea borrar y procede a buscarlo en la lista. Si existe, se muestran sus datos y se pide al usuario una confirmación. En caso de ser aceptada, se elimina el autobús seleccionado usando el método eliminar del TAD.

2.1.3. Modificación de autobús

<u>Método:</u> modificarAutobus()

El programa solicita la matrícula del autobús y, una vez localizado en la lista, se solicitan los nuevos valores y se actualizan mediante los métodos setters. En el caso de que se modifique el número de plazas, el autobús se elimina de la lista y se vuelve a insertar en la posición correcta para mantener el orden.

2.1.4. Registrar viaje

Método: altaViaje()

El usuario introduce la matrícula del autobús que realizará el viaje y, a continuación, los datos del viaje. Se comprueba que el código del viaje no está repetido en toda la estructura y también se comprueba que no exista conflicto de origen, destino y hora en ese autobús. Si las validaciones son correctas, el viaje se inserta en la lista de viajes del autobús seleccionado.

2.1.5. Borrar viaje

Método: bajaViaje()

El usuario introduce la matrícula del autobús al que se le quiere eliminar uno de sus viajes y, una vez localizados autobús y viaje, se solicita una confirmación para el borrado. En caso afirmativo, el viaje se elimina mediante el método eliminar.

2.1.6. Modificación de viaje

Método: modificarViaje()

El programa solicita la matrícula del autobús y, a continuación, muestra todos los viajes que realiza para mostrarlos y que el usuario seleccione el que quiere editar utilizando para ello su código. Posteriormente se solicitan los datos, que se actualizan mediante setters y se validan las ciudades antes de aplicar los cambios.

2.2. Opción 2: Menú de Listados

2.2.1. Listado general de autobuses por número de plazas ascendente

Dado que la lista está ordenada por número de plazas, el programa recorre desde el primero hasta el último de los autobuses de la lista mediante sig y muestra sus datos por pantalla.

2.2.2. Listado general de autobuses por número de plazas descendente

Dado que la lista está ordenada por número de plazas, el programa recorre desde el último hasta el primer autobús de la lista mediante ant y muestra sus datos por pantalla.

2.2.3. Listado general de viajes

El programa se recorre la lista de los autobuses y, en cada uno de ellos, la lista de viajes asociados, de forma que muestra los datos de estos últimos por pantalla.

2.2.4. Listado de viajes que parten de una ciudad determinada
El usuario introduce la ciudad que desea y el programa realiza una búsqueda de todos los viajes que parten de la ciudad seleccionada. Para esto, es necesario recorrer la lista de autobuses y la lista de viajes de cada autobús.

2.2.5. Listado de viajes que llegan a una ciudad determinada
El usuario introduce la ciudad que desea y el programa realiza una búsqueda de todos los viajes cuyo destino es la ciudad seleccionada. Para esto, es necesario recorrer la lista de autobuses y la lista de viajes de cada autobús.

2.2.6. Listado de autobuses que tienen una capacidad mayor o igual a la indicada por el usuario
El usuario introduce el número de plazas que desea y el programa busca en la lista cuales de ellos tienen una capacidad igual o mayor y los muestra por pantalla.

2.2.7. Listado del autobús o autobuses que realizan la mayor cantidad de viajes
El programa recorre la lista de autobuses de forma que en cada uno calcula el número total de viajes que tiene asociados y guarda el valor máximo encontrado. A continuación, realiza un segundo recorrido en la misma lista para mostrar aquellos autobuses cuyo número total de viajes coincide con el calculado en la pasada anterior. Finaliza mostrando por pantalla la matrícula de todos los autobuses encontrados junto con el número de viajes que realizan.

2.2.8. Cantidad total de viajes que realizan los autobuses de la empresa
El programa recorre el listado de autobuses y calcula el número de viajes que realiza cada autobús, sumándolo al total. Para terminar, muestra por pantalla la cantidad calculada.

2.2.9. Cantidad total de pasajeros que viajan de una ciudad a otra
El usuario introduce las ciudades de origen y destino para que el programa recorra la lista de autobuses buscando todos aquellos viajes que se realicen entre las ciudades dadas, considerando que todas las plazas se ocupan, es decir, que el número de viajeros es el número de plazas del autobús. Después suma la cantidad al total y lo muestra por pantalla.

3. LISTADO DEL CÓDIGO FUENTE DESARROLLADO

Se adjunta el código generando:

3.1. Clase Autobus

```
4.  package dominio;
    import librerias.estructurasDeDatos.lineales.LEG;

    public class Autobus {
        private String matricula;
        private int anioCompra;
        private int plazas;
        private LEG<Viaje> viajes; // AHORA ES PRIVADO

        public Autobus(String matricula, int anioCompra, int plazas) {
            // Validaciones defensivas
            if (matricula == null || matricula.trim().isEmpty()) {
                throw new IllegalArgumentException("La matrícula no
    puede estar vacía.");
            }
            if (plazas <= 0) {
                throw new IllegalArgumentException("El número de plazas
    debe ser mayor que 0.");
            }

            this.matricula = matricula;
            this.anioCompra = anioCompra;
            this.plazas = plazas;
            this.viajes = new LEG<>();
        }

        public String getMatricula() { return matricula; }
        public int getAnioCompra() { return anioCompra; }
        public int getPlazas() { return plazas; }

        // NUEVO GETTER PARA LA SUBLISTA
        public LEG<Viaje> getViajes() { return viajes; }

        public void setAnioCompra(int anioCompra) { this.anioCompra =
    anioCompra; }

        public void setPlazas(int plazas) {
            if (plazas <= 0) throw new IllegalArgumentException("Las
    plazas deben ser > 0.");
            this.plazas = plazas;
        }

        @Override
        public boolean equals(Object obj) {
            if (this == obj) return true;
            if (!(obj instanceof Autobus)) return false;
            return this.matricula.equalsIgnoreCase(((Autobus)
    obj).matricula);
        }
    }
```

3.2. Clase Viaje

```
4.  package dominio;

    public class Viaje {
        private int codigo;
        private String origen;
        private String destino;
        private String hora;

        public Viaje(int codigo, String origen, String destino, String
    hora) {
            // Validaciones defensivas
            if (origen == null || destino == null ||
    origen.trim().isEmpty() || destino.trim().isEmpty()) {
                throw new IllegalArgumentException("Las ciudades no
    pueden estar vacías.");
            }
            if (origen.equalsIgnoreCase(destino)) {
                throw new IllegalArgumentException("Las ciudades de
    origen y destino deben ser distintas.");
            }

            this.codigo = codigo;
            this.origen = origen;
            this.destino = destino;
            this.hora = hora;
        }

        public int getCodigo() { return codigo; }
        public String getOrigen() { return origen; }
        public String getDestino() { return destino; }
        public String getHora() { return hora; }

        public void setOrigen(String origen) { this.origen = origen; }
        public void setDestino(String destino) { this.destino =
    destino; }
        public void setHora(String hora) { this.hora = hora; }

        @Override
        public boolean equals(Object obj) {
            if (this == obj) return true;
            if (!(obj instanceof Viaje)) return false;
            return this.codigo == ((Viaje) obj).codigo;
        }

        @Override
        public String toString() {
            return String.format("Viaje %d: %s -> %s (%s)", codigo,
    origen, destino, hora);
        }
    }
```

3.3. Clase GestionAutobus

```
4.  package dominio;

    import librerias.estructurasDeDatos.lineales.LEG;
    import librerias.estructurasDeDatos.lineales.NodoLEG;
```

```
import java.util.Scanner;

public class GestionBus {
    static LEG<Autobus> empresa = new LEG<>();
    static Scanner sc = new Scanner(System.in);

    // MENÚS SECUNDARIOS
    private static void menuMantenimiento() {
        int op = -1;
        do {
            System.out.println("\nMENÚ MANTENIMIENTO");
            System.out.println("1.- Alta de autobús");
            System.out.println("2.- Baja de autobús");
            System.out.println("3.- Modificación de autobús");
            System.out.println("4.- Registrar viaje");
            System.out.println("5.- Borrar viaje");
            System.out.println("6.- Modificar viaje");
            System.out.println("0.- Volver al menú principal");
            System.out.print("Opción: ");
            try {
                op = Integer.parseInt(sc.nextLine());
                if (op == 1) altaAutobus();
                else if (op == 2) bajaAutobus();
                else if (op == 3) modificarAutobus();
                else if (op == 4) altaViaje();
                else if (op == 5) bajaViaje();
                else if (op == 6) modificarViaje();
            } catch (Exception e) {
                System.out.println("Error: Introduzca un número
válido.");
            }
        } while (op != 0);
    }

    public static void main(String[] args) {
        System.out.println("Bienvenido a la aplicación GEST_BUS");
        int opcion = -1;
        do {
            System.out.println("\nMENÚ PRINCIPAL");
            System.out.println("1.- Mantenimiento");
            System.out.println("2.- Listados");
            System.out.println("0.- Salir");
            System.out.print("Opción: ");
            try {
                opcion = Integer.parseInt(sc.nextLine());
                if (opcion == 1) menuMantenimiento();
                else if (opcion == 2) menuListados();
            } catch (Exception e) {
                System.out.println("Error: Introduzca un número
válido.");
            }
        } while (opcion != 0);
        System.out.println("Gracias por utilizar la aplicación
GEST_BUS");
    }

    private static void menuListados() {
        int op = -1;
        do {
            System.out.println("\nMENÚ LISTADOS");
            System.out.println("1.- General de autobuses (orden
```

```
ascendente por plazas)");
            System.out.println("2.- General de autobuses (orden
descendente por plazas)");
            System.out.println("3.- Listado general de viajes");
            System.out.println("4.- Listado de viajes que parten de
una ciudad determinada");
            System.out.println("5.- Listado de viajes que llegan a
una ciudad determinada");
            System.out.println("6.- Listado de autobuses con
capacidad mayor o igual a la indicada");
            System.out.println("7.- Listado del autobús o autobuses
que realizan la mayor cantidad de viajes");
            System.out.println("8.- Cantidad total de viajes que
realizan los autobuses");
            System.out.println("9.- Cantidad total de pasajeros que
viajan de una ciudad a otra");
            System.out.println("0.- Volver al menú principal");
            System.out.print("Opción: ");
            try {
                op = Integer.parseInt(sc.nextLine());
                if (op >= 1 && op <= 9) procesarListado(op);
            } catch (Exception e) {
                System.out.println("Error: Introduzca un número
válido.");
            }
        } while (op != 0);
    }

    // OPERACIONES DE MANTENIMIENTO
    private static void altaAutobus() {
        String resp;
        do {
            System.out.print("Introduzca matrícula: ");
            String mat = sc.nextLine();
            Autobus dummy = new Autobus(mat, 1, 1); // Pasamos 1 en
vez de 0 para sortear las validaciones del constructor

            if (empresa.buscar(dummy) != null) {
                System.out.println("Error: ya existe un autobús
registrado con esa matrícula");
            } else {
                try {
                    System.out.print("Año de compra: ");
                    int anio = Integer.parseInt(sc.nextLine());
                    System.out.print("Número de plazas: ");
                    int plazas = Integer.parseInt(sc.nextLine());

                    Autobus nuevo = new Autobus(mat, anio, plazas);
                    insertarAutobusOrdenadoPorPlazas(nuevo);
                    System.out.println("Autobús añadido.");
                } catch (IllegalArgumentException e) {
                    System.out.println("Error en los datos del
autobús: " + e.getMessage());
                }
            }
            System.out.print("¿Desea añadir un nuevo autobús?
(S/N): ");
            resp = sc.nextLine();
        } while (resp.equalsIgnoreCase("S"));
    }
```

```
    private static void bajaAutobus() {
        String resp;
        do {
            System.out.print("Introduzca matrícula a dar de baja:
");
            String mat = sc.nextLine();
            Autobus bus = empresa.buscar(new Autobus(mat, 1, 1));
// Pasamos 1 para la búsqueda

            if (bus == null) {
                System.out.println("Error: esta matricula no está
registrada en la aplicación");
            } else {
                System.out.println("Datos: Matrícula " +
bus.getMatricula() + " | Año: " + bus.getAnioCompra() + " | Plazas:
" + bus.getPlazas());
                System.out.print("¿Desea realmente dar de baja a
este autobús? (S/N): ");
                String conf = sc.nextLine();
                if (conf.equalsIgnoreCase("S")) {
                    empresa.eliminar(bus);
                    System.out.println("Baja registrada");
                } else {
                    System.out.println("Proceso de baja abortado");
                }
            }
            System.out.print("¿Desea dar de baja otro autobús?
(S/N): ");
            resp = sc.nextLine();
        } while (resp.equalsIgnoreCase("S"));
    }

    private static void modificarAutobus() {
        String resp;
        do {
            System.out.print("Introduzca matrícula: ");
            String mat = sc.nextLine();
            Autobus bus = empresa.buscar(new Autobus(mat, 1, 1));
// Pasamos 1 para la búsqueda

            if (bus == null) {
                System.out.println("Error: esta matricula no está
registrada en la aplicación");
            } else {
                try {
                    System.out.print("Nuevo año de compra (actual "
+ bus.getAnioCompra() + "): ");

bus.setAnioCompra(Integer.parseInt(sc.nextLine()));
                    System.out.print("Nuevo número de plazas
(actual " + bus.getPlazas() + "): ");
                    int nuevasPlazas =
Integer.parseInt(sc.nextLine());

                    // Si se cambian las plazas, hay que reordenar
el autobús en la lista.
                    if (nuevasPlazas != bus.getPlazas()) {
                        empresa.eliminar(bus);
                        bus.setPlazas(nuevasPlazas);
                        insertarAutobusOrdenadoPorPlazas(bus);
                    }
```

```
                        System.out.println("Autobús modificado
correctamente.");
                    } catch (IllegalArgumentException e) {
                        System.out.println("Error: " + e.getMessage());
                    }
                }
                System.out.print("¿Desea modificar los datos de otro
autobús? (S/N): ");
                resp = sc.nextLine();
            } while (resp.equalsIgnoreCase("S"));
        }

        private static void altaViaje() {
            String resp;
            do {
                System.out.print("Matrícula del autobús que realizará
el viaje: ");
                String mat = sc.nextLine();
                Autobus bus = empresa.buscar(new Autobus(mat, 1, 1));
// Pasamos 1 para la búsqueda

                if (bus == null) {
                    System.out.println("Error: esta matrícula no está
registrada.");
                } else {
                    try {
                        System.out.print("Código del viaje: ");
                        int codigo = Integer.parseInt(sc.nextLine());

                        if (existeCodigoViajeGlobal(codigo)) {
                            System.out.println("Error: El código de
viaje ya existe en la aplicación.");
                        } else {
                            System.out.print("Ciudad de origen
(Madrid/Segovia/Barcelona/Sevilla): ");
                            String origen = sc.nextLine();
                            System.out.print("Ciudad de destino: ");
                            String destino = sc.nextLine();
                            System.out.print("Hora del viaje (ej:
16:30): ");
                            String hora = sc.nextLine();

                            if (!ciudadesValidas(origen, destino)) {
                                System.out.println("Error: Ciudades
inválidas o coinciden.");
                            } else if (conflictoViajeEnBus(bus, origen,
destino, hora)) {
                                System.out.println("Error: El autobús
ya realiza un viaje similar (mismo origen, destino y hora).");
                            } else {
                                bus.getViajes().insertar(new
Viaje(codigo, origen, destino, hora)); // CAMBIO AQUÍ
                                System.out.println("Viaje registrado
con éxito.");
                            }
                        }
                    } catch (IllegalArgumentException e) {
                        System.out.println("Error: " + e.getMessage());
                    }
                }
                System.out.print("¿Desea registrar otro viaje? (S/N):
```

```
");
            resp = sc.nextLine();
        } while (resp.equalsIgnoreCase("S"));
    }

    private static void bajaViaje() {
        System.out.print("Matrícula del autobús: ");
        String mat = sc.nextLine();
        Autobus bus = empresa.buscar(new Autobus(mat, 1, 1));

        if (bus != null) {
            System.out.print("Código del viaje a borrar: ");
            int cod = Integer.parseInt(sc.nextLine());
            Viaje v = bus.getViajes().buscar(new Viaje(cod, "a",
"b", "")); // CAMBIO AQUÍ

            if (v != null) {
                System.out.println("Datos del viaje: " + v);
                System.out.print("¿Confirmar borrado? (S/N): ");
                if (sc.nextLine().equalsIgnoreCase("S")) {
                    bus.getViajes().eliminar(v); // CAMBIO AQUÍ
                    System.out.println("Viaje borrado.");
                } else {
                    System.out.println("Borrado cancelado.");
                }
            } else {
                System.out.println("Error: Viaje no encontrado en
este autobús.");
            }
        } else {
            System.out.println("Error: Autobús no encontrado.");
        }
    }

    private static void modificarViaje() {
        System.out.print("Matrícula del autobús: ");
        String mat = sc.nextLine();
        Autobus bus = empresa.buscar(new Autobus(mat, 1, 1));

        if (bus != null && !bus.getViajes().esVacia()) { // CAMBIO
AQUÍ
            System.out.println("Viajes del autobús:");
            NodoLEG<Viaje> aux = bus.getViajes().primero; // CAMBIO
AQUÍ
            while(aux != null){
                System.out.println(aux.dato);
                aux = aux.sig;
            }

            System.out.print("¿Qué viaje desea modificar?
(Introduzca código): ");
            int cod = Integer.parseInt(sc.nextLine());
            Viaje v = bus.getViajes().buscar(new Viaje(cod, "a",
"b", "")); // CAMBIO AQUÍ

            if (v != null) {
                System.out.print("Nuevo origen: "); String ori =
sc.nextLine();
                System.out.print("Nuevo destino: "); String des =
sc.nextLine();
                System.out.print("Nueva hora: "); String hora =
```

```
    sc.nextLine();

                    if (ciudadesValidas(ori, des)) {
                        v.setOrigen(ori);
                        v.setDestino(des);
                        v.setHora(hora);
                        System.out.println("Autobús " + mat + ": Viaje
" + cod + " modificado correctamente");
                    } else {
                        System.out.println("Error: Ciudades no
válidas.");
                    }
                } else {
                    System.out.println("Viaje no encontrado.");
                }
            } else {
                System.out.println("Error: Autobús no encontrado o no
tiene viajes.");
            }
        }

        // ==========================================
        // OPERACIONES DE LISTADOS
        // ==========================================
        private static void procesarListado(int op) {
            switch (op) {
                case 1:
                    System.out.println("\nLISTADO GENERAL DE AUTOBUSES
POR NÚMERO DE PLAZAS (Orden Ascendente)");
                    System.out.printf("%-15s %-15s %-10s\n",
"Matrícula", "Año Compra", "Plazas");
                    System.out.println("------------------------------
--------------");
                    NodoLEG<Autobus> aux1 = empresa.primero;
                    while (aux1 != null) {
                        System.out.printf("%-15s %-15d %-10d\n",
aux1.dato.getMatricula(), aux1.dato.getAnioCompra(),
aux1.dato.getPlazas());
                        aux1 = aux1.sig;
                    }
                    break;
                case 2:
                    System.out.println("\nLISTADO GENERAL DE AUTOBUSES
POR NÚMERO DE PLAZAS (Orden Descendente)");
                    System.out.printf("%-15s %-15s %-10s\n",
"Matrícula", "Año Compra", "Plazas");
                    System.out.println("------------------------------
--------------");
                    NodoLEG<Autobus> aux2 = empresa.ultimo;
                    while (aux2 != null) {
                        System.out.printf("%-15s %-15d %-10d\n",
aux2.dato.getMatricula(), aux2.dato.getAnioCompra(),
aux2.dato.getPlazas());
                        aux2 = aux2.ant;
                    }
                    break;
                case 3:
                    System.out.println("\nLISTADO GENERAL DE VIAJES");
                    NodoLEG<Autobus> a3 = empresa.primero;
                    while (a3 != null) {
                        System.out.println("Autobús: " +
```

```
a3.dato.getMatricula());
                    NodoLEG<Viaje> v3 =
a3.dato.getViajes().primero; // CAMBIO AQUÍ
                    if(v3 == null) System.out.println("  Sin viajes
asignados.");
                    while(v3 != null) {
                        System.out.println("  - " + v3.dato);
                        v3 = v3.sig;
                    }
                    a3 = a3.sig;
                }
                break;
            case 4:
                System.out.print("Introduzca ciudad de partida: ");
                String origen = sc.nextLine();
                System.out.println("\nVIAJES QUE PARTEN DE " +
origen.toUpperCase());
                NodoLEG<Autobus> a4 = empresa.primero;
                while (a4 != null) {
                    NodoLEG<Viaje> v4 =
a4.dato.getViajes().primero; // CAMBIO AQUÍ
                    while(v4 != null) {
                        if
(v4.dato.getOrigen().equalsIgnoreCase(origen)) {
                            System.out.println("Bus: " +
a4.dato.getMatricula() + " | " + v4.dato);
                        }
                        v4 = v4.sig;
                    }
                    a4 = a4.sig;
                }
                break;
            case 5:
                System.out.print("Introduzca ciudad de llegada: ");
                String destino = sc.nextLine();
                System.out.println("\nVIAJES QUE LLEGAN A " +
destino.toUpperCase());
                NodoLEG<Autobus> a5 = empresa.primero;
                while (a5 != null) {
                    NodoLEG<Viaje> v5 =
a5.dato.getViajes().primero; // CAMBIO AQUÍ
                    while(v5 != null) {
                        if
(v5.dato.getDestino().equalsIgnoreCase(destino)) {
                            System.out.println("Bus: " +
a5.dato.getMatricula() + " | " + v5.dato);
                        }
                        v5 = v5.sig;
                    }
                    a5 = a5.sig;
                }
                break;
            case 6:
                System.out.print("Introduzca capacidad mínima: ");
                int cap = Integer.parseInt(sc.nextLine());
                System.out.println("\nAUTOBUSES CON " + cap + " O
MÁS PLAZAS");
                NodoLEG<Autobus> a6 = empresa.primero;
                while (a6 != null) {
                    if (a6.dato.getPlazas() >= cap) {
                        System.out.println("Matrícula: " +
```

```
a6.dato.getMatricula() + " | Plazas: " + a6.dato.getPlazas());
                    }
                    a6 = a6.sig;
                }
                break;
            case 7:
                System.out.println("\nAUTOBUSES CON MAYOR CANTIDAD
DE VIAJES");
                int max = -1;
                NodoLEG<Autobus> a7 = empresa.primero;
                while (a7 != null) {
                    if (a7.dato.getViajes().talla() > max) max =
a7.dato.getViajes().talla(); // CAMBIO AQUÍ
                    a7 = a7.sig;
                }
                if (max > 0) {
                    a7 = empresa.primero;
                    while (a7 != null) {
                        if (a7.dato.getViajes().talla() == max) {
// CAMBIO AQUÍ
                            System.out.println("Matrícula: " +
a7.dato.getMatricula() + " | Viajes: " + max);
                        }
                        a7 = a7.sig;
                    }
                } else {
                    System.out.println("Ningún autobús tiene
viajes.");
                }
                break;
            case 8:
                int totalViajes = 0;
                NodoLEG<Autobus> a8 = empresa.primero;
                while (a8 != null) {
                    totalViajes += a8.dato.getViajes().talla(); //
CAMBIO AQUÍ
                    a8 = a8.sig;
                }
                System.out.println("\nCANTIDAD TOTAL DE VIAJES DE
LA EMPRESA: " + totalViajes);
                break;
            case 9:
                System.out.print("Ciudad origen: "); String ori =
sc.nextLine();
                System.out.print("Ciudad destino: "); String des =
sc.nextLine();
                int totalPasajeros = 0;
                NodoLEG<Autobus> a9 = empresa.primero;
                while (a9 != null) {
                    NodoLEG<Viaje> v9 =
a9.dato.getViajes().primero; // CAMBIO AQUÍ
                    while(v9 != null) {
                        if
(v9.dato.getOrigen().equalsIgnoreCase(ori) &&
v9.dato.getDestino().equalsIgnoreCase(des)) {
                            totalPasajeros += a9.dato.getPlazas();
                        }
                        v9 = v9.sig;
                    }
                    a9 = a9.sig;
                }
```

```
                    System.out.println("\nTOTAL PASAJEROS ESTIMADOS
ENTRE " + ori.toUpperCase() + " Y " + des.toUpperCase() + ": " +
totalPasajeros);
                    break;
            }
        }

        // ==========================================
        // MÉTODOS AUXILIARES Y DE VALIDACIÓN
        // ==========================================

        private static void insertarAutobusOrdenadoPorPlazas(Autobus
nuevo) {
            if (empresa.esVacia()) {
                empresa.insertar(nuevo);
                return;
            }

            NodoLEG<Autobus> aux = empresa.primero;
            while (aux != null && aux.dato.getPlazas() <
nuevo.getPlazas()) {
                aux = aux.sig;
            }

            NodoLEG<Autobus> nodoNuevo = new NodoLEG<>(nuevo);
            if (aux == empresa.primero) {
                nodoNuevo.sig = empresa.primero;
                empresa.primero.ant = nodoNuevo;
                empresa.primero = nodoNuevo;
            } else if (aux == null) {
                nodoNuevo.ant = empresa.ultimo;
                empresa.ultimo.sig = nodoNuevo;
                empresa.ultimo = nodoNuevo;
            } else {
                nodoNuevo.sig = aux;
                nodoNuevo.ant = aux.ant;
                aux.ant.sig = nodoNuevo;
                aux.ant = nodoNuevo;
            }
        }

        private static boolean existeCodigoViajeGlobal(int codigo) {
            NodoLEG<Autobus> auxBus = empresa.primero;
            while (auxBus != null) {
                if (auxBus.dato.getViajes().buscar(new Viaje(codigo,
"a", "b", "")) != null) { // CAMBIO AQUÍ
                    return true;
                }
                auxBus = auxBus.sig;
            }
            return false;
        }

        private static boolean ciudadesValidas(String origen, String
destino) {
            if (origen.equalsIgnoreCase(destino)) return false;
            String[] permitidas = {"Madrid", "Segovia", "Barcelona",
"Sevilla"};
            boolean oriValido = false, desValido = false;
            for (String c : permitidas) {
                if (c.equalsIgnoreCase(origen)) oriValido = true;
```

```
                if (c.equalsIgnoreCase(destino)) desValido = true;
            }
            return oriValido && desValido;
        }

        private static boolean conflictoViajeEnBus(Autobus bus, String
    origen, String destino, String hora) {
            NodoLEG<Viaje> aux = bus.getViajes().primero; // CAMBIO
    AQUÍ
            while (aux != null) {
                if (aux.dato.getOrigen().equalsIgnoreCase(origen) &&
                        aux.dato.getDestino().equalsIgnoreCase(destino)
    &&
                        aux.dato.getHora().equalsIgnoreCase(hora)) {
                    return true;
                }
                aux = aux.sig;
            }
            return false;
        }
    }
```

3.4. Clase LEG

```
4.  package librerias.estructurasDeDatos.lineales;
    import librerias.estructurasDeDatos.modelos.I_LE;

    public class LEG<E> implements I_LE<E> {
        public NodoLEG<E> primero;
        public NodoLEG<E> ultimo;
        private int talla;

        public LEG() {
            this.primero = null;
            this.ultimo = null;
            this.talla = 0;
        }

        @Override
        public void insertar(E e) {
            NodoLEG<E> nuevo = new NodoLEG<>(e);
            if (esVacia()) {
                primero = ultimo = nuevo;
            } else {
                ultimo.sig = nuevo;
                nuevo.ant = ultimo;
                ultimo = nuevo;
            }
            talla++;
        }

        @Override
        public boolean eliminar(E e) {
            NodoLEG<E> aux = primero;
            while (aux != null && !aux.dato.equals(e)) {
                aux = aux.sig;
            }
            if (aux != null) {
                if (aux == primero) primero = aux.sig;
```

```
                else aux.ant.sig = aux.sig;

                if (aux == ultimo) ultimo = aux.ant;
                else aux.sig.ant = aux.ant;

                talla--;
                return true;
            }
            return false;
        }

        @Override
        public E buscar(E e) {
            NodoLEG<E> aux = primero;
            while (aux != null) {
                if (aux.dato.equals(e)) return aux.dato;
                aux = aux.sig;
            }
            return null;
        }

        @Override
        public boolean esVacia() { return talla == 0; }

        @Override
        public int talla() { return talla; }
    }
```

3.5. Clase NodoLEG

```
4.  package librerias.estructurasDeDatos.lineales;

    public class NodoLEG<E> {
        public E dato;
        public NodoLEG<E> sig;
        public NodoLEG<E> ant;

        public NodoLEG(E dato) {
            this.dato = dato;
            this.sig = null;
            this.ant = null;
        }
    }
```

3.6. Clase I_LE

```
4.  package librerias.estructurasDeDatos.modelos;

    public interface I_LE<E> {
        void insertar(E e);
        boolean eliminar(E e);
        E buscar(E e);
        boolean esVacia();
        int talla();
    }
```

4. PRUEBAS DE EJECUCIÓN

Se adjuntan capturas de la ejecución del código desarrollado:

```
Bienvenido a la aplicación GEST_BUS

MENÚ PRINCIPAL
1.- Mantenimiento
2.- Listados
0.- Salir
Opción:
```

```
MENÚ MANTENIMIENTO
1.- Alta de autobús
2.- Baja de autobús
3.- Modificación de autobús
4.- Registrar viaje
5.- Borrar viaje
6.- Modificar viaje
0.- Volver al menú principal
Opción:
```

```
Opción: 1
Introduzca matrícula: 1234ABC
Año de compra: 2023
Número de plazas: 55
Autobús añadido.
¿Desea añadir un nuevo autobús? (S/N): s
Introduzca matrícula: 1235CDC
Año de compra: 2020
Número de plazas: 49
Autobús añadido.
¿Desea añadir un nuevo autobús? (S/N): N
```

```
Opción: 2
Introduzca matrícula a dar de baja: 1235CDC
Datos: Matrícula 1235CDC | Año: 2020 | Plazas: 52
¿Desea realmente dar de baja a este autobús? (S/N): S
Baja registrada
¿Desea dar de baja otro autobús? (S/N): N
```

```
Opción: 3
Introduzca matrícula: 1235CDC
Nuevo año de compra (actual 2020): 2020
Nuevo número de plazas (actual 49): 52
Autobús modificado correctamente.
¿Desea modificar los datos de otro autobús? (S/N): n
```

```
Opción: 4
Matrícula del autobús que realizará el viaje: 1234ABC
Código del viaje: 001
Ciudad de origen (Madrid/Segovia/Barcelona/Sevilla): Segovia
Ciudad de destino: Madrid
Hora del viaje (ej: 16:30): 10:00
Viaje registrado con éxito.
¿Desea registrar otro viaje? (S/N): n
```

```
Opción: 5
Matrícula del autobús: 1235CDC
Código del viaje a borrar: 2
Datos del viaje: Viaje 2: Madrid -> Barcelona (12:15)
¿Confirmar borrado? (S/N): S
Viaje borrado.
```

```
Opción: 6
Matrícula del autobús: 1234ABC
Viajes del autobús:
Viaje 1: Segovia -> Madrid (10:00)
¿Qué viaje desea modificar? (Introduzca código): 1
Nuevo origen: Segovia
Nuevo destino: Sevilla
Nueva hora: 10:00
Autobús 1234ABC: Viaje 1 modificado correctamente
```

```
MENÚ LISTADOS
1.- General de autobuses (orden ascendente por plazas)
2.- General de autobuses (orden descendente por plazas)
3.- Listado general de viajes
4.- Listado de viajes que parten de una ciudad determinada
5.- Listado de viajes que llegan a una ciudad determinada
6.- Listado de autobuses con capacidad mayor o igual a la indicada
7.- Listado del autobús o autobuses que realizan la mayor cantidad de viajes
8.- Cantidad total de viajes que realizan los autobuses
9.- Cantidad total de pasajeros que viajan de una ciudad a otra
0.- Volver al menú principal
Opción:
```

```
Opción: 1

LISTADO GENERAL DE AUTOBUSES POR NÚMERO DE PLAZAS (Orden Ascendente)
Matrícula       Año Compra      Plazas
--------------------------------------------
1235CDC         2020            52
1234ABC         2023            55
```

```
Opción: 2

LISTADO GENERAL DE AUTOBUSES POR NÚMERO DE PLAZAS (Orden Descendente)
Matrícula       Año Compra      Plazas
--------------------------------------------
1234ABC         2023            55
1235CDC         2020            52
```

```
Opción: 3

LISTADO GENERAL DE VIAJES
Autobús: 1235CDC
  - Viaje 2: Madrid -> Barcelona (12:15)
Autobús: 1234ABC
  - Viaje 1: Segovia -> Sevilla (10:00)
```

```
Opción: 4
Introduzca ciudad de partida: Segovia

VIAJES QUE PARTEN DE SEGOVIA
Bus: 1234ABC | Viaje 1: Segovia -> Sevilla (10:00)
```

```
Opción: 5
Introduzca ciudad de llegada: Barcelona

VIAJES QUE LLEGAN A BARCELONA
Bus: 1235CDC | Viaje 2: Madrid -> Barcelona (12:15)
```

```
Opción: 6
Introduzca capacidad mínima: 50

AUTOBUSES CON 50 O MÁS PLAZAS
Matrícula: 1235CDC | Plazas: 52
Matrícula: 1234ABC | Plazas: 55
```

```
Opción: 7

AUTOBUSES CON MAYOR CANTIDAD DE VIAJES
Matrícula: 1235CDC | Viajes: 1
Matrícula: 1234ABC | Viajes: 1
```

```
Opción: 8

CANTIDAD TOTAL DE VIAJES DE LA EMPRESA: 2
```

```
Opción: 9
Ciudad origen: Segovia
Ciudad destino: Sevilla

TOTAL PASAJEROS ESTIMADOS ENTRE SEGOVIA Y SEVILLA: 55
```